

EMBEDDED LINUX SYSTEMS

LAB-03: Debugging Kernel & Embedded Application Performance Analysis

Student	Nguyen Dang Quang
Student ID	SE201072
Target	ARMv7-A Cortex-A9 / QEMU vexpress-a9
Subject	GDB Remote Debugging • Valgrind • strace / perf • cyclicttest
Report	Experimental results and technical analysis

This document reports measured results from the completed laboratory sessions. The screenshots, raw GDB transcript, profiling logs and histogram data are preserved separately under the LAB03 submission folder.

Report date: 10 October 2026.

1. System Information and Experiment Scope

Ubuntu 24.04 LTS x86-64 host; GDB multiarch 15.1; Valgrind 3.22.0; perf 7.0.14; strace 6.8; cyclicttest 2.50; QEMU system ARM 8.2.2. The guest ARM kernel was built in LAB01 (Linux 5.15.0).

The host reported **7.0.0-38-generic** when the report environment was recorded. The saved cyclicttest measurements were performed earlier on **7.0.0-34-generic (PREEMPT_DYNAMIC)**. Both are general-purpose host kernels, not PREEMPT_RT.

Experiments: remote userspace debugging on QEMU ARM; host Valgrind checking and ARM VmRSS observation; host perf and host/guest strace; cyclicttest under idle-like and stressed host conditions. No PREEMPT_RT measurements were performed.

2. Part A — GDB Remote Debugging

A remote gdbserver on the ARM QEMU target listened on TCP port 1234. GDB multiarch on the host loaded the ARM ELF with debugging symbols and connected using `target remote localhost:1234`. All three intentional bugs were investigated in the same GDB session.

Bug	Evidence (ARM GDB)	Root cause and correction
Array out-of-bounds	<code>process_data</code> , <code>buggy_app.c:9</code> ; <code>len=5</code> ; <code>i=5</code> ; <code>sum=150</code> ; <code>backtrace to main:35</code>	Loop checks <code>i <= len</code> ; element <code>data[5]</code> is outside an array of five elements. Replace with <code>i < len</code> .
Use-after-free	<code>create_message</code> , <code>buggy_app.c:18–19</code> ; returned buffer address <code>0x404198</code> ; <code>main</code> retains <code>0x404198</code>	<code>free(buf)</code> is called before return <code>buf</code> . Transfer ownership to caller, then <code>free(msg)</code> only after its final use.
Signed integer overflow	<code>calculate</code> , <code>buggy_app.c:24–25</code> ; <code>100000 × 100000</code> gives observed int <code>1410065408</code> ; correct math <code>100000000000</code> ; <code>sizeof(int)=4</code>	Multiplication exceeds signed 32-bit range (undefined C behavior). Widen an operand before multiplication: <code>long long r = (long long)a * b</code> .

```

Breakpoint 3, process_data (data=0x7efffcf8, len=5) at buggy_app.c:9
    sum += data[i];
$3 = 5
$4 = 5
$5 = 150
#0 process_data (data=0x7efffcf8, len=5) at buggy_app.c:9
#1 0x00400774 in main (argc=1, argv=0x7efffe94) at buggy_app.c:35
Breakpoint 4 at 0x4006cc: file buggy_app.c, line 16.
Continuing.

```

Figure A1. Array boundary and GDB backtrace.

```

0 create_message (text=0x400940 "Hello Embedded Linux") at buggy_app.c:19
1 0x00400788 in main (argc=1, argv=0x7efffe94) at buggy_app.c:39
2 until exit from #0 create_message (text=0x400940 "Hello Embedded Linux") at buggy_app.c:19
3 0x00400788 in main (argc=1, argv=0x7efffe94) at buggy_app.c:39
4 char *msg = create_message("Hello Embedded Linux");
5 value returned is $10 = 0x404198 "\004\004"
6 $11 = 0x64 error: Cannot access memory at address 0x64
7 /* Test 2 */
8 printf("[TEST2] Creating message\n");
9 char *msg = create_message("Hello Embedded Linux");
10 printf("Message: %s\n", msg);
11
12 /* Test 3 */
13 printf("[TEST3] Integer calculation\n");
14 printf("Message: %s\n", msg);
15 $12 = 0x404198
16 main (argc=1, argv=0x7efffe94) at buggy_app.c:40
17 breakpoint 5 at 0x400702: file buggy_app.c, line 24.
18 continuing.
19
20 breakpoint 5, calculate (a=100000, b=100000) at buggy_app.c:24
21 int result = a * b;
22 $13 = 100000
23 $14 = 100000
24 /* Bug 3: Integer overflow */
25 int calculate(int a, int b) {
26     int result = a * b;
27     printf("calculate(%d, %d) = %d\n", a, b, result);
28     return result;
29 }
30 printf("calculate(%d, %d) = %d\n", a, b, result);
31 $15 = 1410065408
32 $16 = 10000000000
33 $17 = 4
34 calculate (a=100000, b=100000) at buggy_app.c:25
35 0x004007b4 in main (argc=1, argv=0x7efffe94) at buggy_app.c:44
36 0x186a0 100000
37 0x186a0 100000
38 0x186a0 100000
39 0x540be400 1410065408
40 0x7efffd08 2130705672
41 0x40099c 4196764
42 0x401eac 4202156
43 0x7efffd0 2130705616
44 0x400729 4196137
45 0x76ffcd8 1996483800
46 0x0 0
47 0x401eac 4202156
48 0x4 4
49 0x7efffd0 0x7efffd0
50 0x4007b5 4196277
51 0x40070c 0x40070c <calculate+20>
52 pcscr 0x200f0030 537854000
53 0x0 0
54 END

```

Figure A2. Freed buffer address returned to the caller.

```

nguyendangquang@nguyendangquang-Bellator:~$ grep -A 45 "Breakpoint 5, calculate" ~/embedded_lab3/logs/gdb_session.txt
Breakpoint 5, calculate (a=100000, b=100000) at buggy_app.c:24
24     int result = a * b;
$13 = 100000
$14 = 100000
22 /* Bug 3: Integer overflow */
23 int calculate(int a, int b) {
24     int result = a * b;
25     printf("calculate(%d, %d) = %d\n", a, b, result);
26     return result;
27 }
25     printf("calculate(%d, %d) = %d\n", a, b, result);
$15 = 1410065408
$16 = 10000000000
$17 = 4
#0 calculate (a=100000, b=100000) at buggy_app.c:25
#1 0x004007b4 in main (argc=1, argv=0x7efffe94) at buggy_app.c:44
r0      0x186a0 100000
r1      0x186a0 100000
r2      0x186a0 100000
r3      0x540be400 1410065408
r4      0x7efffd08 2130705672
r5      0x40099c 4196764
r6      0x401eac 4202156
r7      0x7efffd0 2130705616
r8      0x400729 4196137
r9      0x76ffcd8 1996483800
r10     0x0 0
r11     0x401eac 4202156
r12     0x4 4
sp      0x7efffd0 0x7efffd0
lr      0x4007b5 4196277
pc      0x40070c 0x40070c <calculate+20>
cpsr    0x200f0030 537854000
fpscr   0x0 0
nguyendangquang@nguyendangquang-Bellator:~$

```

Figure A3. Overflow values, backtrace and ARM registers.

GDB transcript — supplied experiment log

The original session transcript is included in full below as text, including breakpoints, expressions, stack traces and register dump. GDB values recorded as \$N are debugger convenience-variable output from the experiment.

```
Remote debugging using localhost:1234
Reading symbols from /home/nguyendangquang/embedded_lab3/rootfs/initramfs/lib/ld-linux-armhf.so.3...

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
Downloading separate debug info for system-supplied DSO at 0x76fe0000

0x76ff0598 in _start () from /home/nguyendangquang/embedded_lab3/rootfs/initramfs/lib/ld-linux-armhf.so.3
Already logging to /home/nguyendangquang/embedded_lab3/logs/gdb_session.txt.
Breakpoint 1 at 0x400732: file buggy_app.c, line 29.
Breakpoint 2 at 0x400682: file buggy_app.c, line 7.
Continuing.
Downloading separate debug info for /home/nguyendangquang/embedded_lab3/rootfs/initramfs/lib/libc.so.6
```

```
Breakpoint 1, main (argc=1, argv=0x7efffe94) at buggy_app.c:29
29 int main(int argc, char *argv[]) {
24     int result = a * b;
25     printf("calculate(%d, %d) = %d\n", a, b, result);
26     return result;
27 }
28
29 int main(int argc, char *argv[]) {
30     printf("=== Buggy App v1.0 - LAB-03 Debug Target ===\n");
31
32     /* Test 1 */
33     int arr[5] = {10, 20, 30, 40, 50};
Num      Type           Disp Enb Address      What
1        breakpoint     keep y   0x00400732 in main at buggy_app.c:29
        breakpoint already hit 1 time
2        breakpoint     keep y   0x00400682 in process_data at buggy_app.c:7
Continuing.
```

```
Breakpoint 2, process_data (data=0x7efffcf8, len=5) at buggy_app.c:7
7     int sum = 0;
$1 = 5
$2 = {10, 20, 30, 40, 50}
#0 process_data (data=0x7efffcf8, len=5) at buggy_app.c:7
#1 0x00400774 in main (argc=1, argv=0x7efffe94) at buggy_app.c:35
5 /* Bug 1: Array out-of-bounds */
6 void process_data(int *data, int len) {
7     int sum = 0;
8     for (int i = 0; i <= len; i++) {
9         sum += data[i];
10    }
11    printf("Sum = %d\n", sum);
12 }
Breakpoint 3 at 0x40068c: file buggy_app.c, line 9.
Continuing.
```

```
Breakpoint 3, process_data (data=0x7efffcf8, len=5) at buggy_app.c:9
9     sum += data[i];
$3 = 5
$4 = 5
$5 = 150
#0 process_data (data=0x7efffcf8, len=5) at buggy_app.c:9
#1 0x00400774 in main (argc=1, argv=0x7efffe94) at buggy_app.c:35
Breakpoint 4 at 0x4006cc: file buggy_app.c, line 16.
Continuing.
```

```
Breakpoint 4, create_message (text=0x400940 "Hello Embedded Linux") at buggy_app.c:16
16 char *buf = malloc(strlen(text) + 1);
14 /* Bug 2: Use after free */
15 char *create_message(const char *text) {
16     char *buf = malloc(strlen(text) + 1);
17     strcpy(buf, text);
18     free(buf);
```

```

19     return buf;
20 }
21
#0 create_message (text=0x400940 "Hello Embedded Linux") at buggy_app.c:16
#1 0x00400788 in main (argc=1, argv=0x7efffe94) at buggy_app.c:39
$6 = 0x400940 "Hello Embedded Linux"
17     strcpy(buf, text);
$7 = 0x404198 ""
18     free(buf);
$8 = 0x404198 "Hello Embedded Linux"
0x404198: "Hello Embedded Linux"
19     return buf;
Undefined command: "19". Try "help".
$9 = 0x404198 "\004\004"
#0 create_message (text=0x400940 "Hello Embedded Linux") at buggy_app.c:19
#1 0x00400788 in main (argc=1, argv=0x7efffe94) at buggy_app.c:39
Run till exit from #0 create_message (text=0x400940 "Hello Embedded Linux") at buggy_app.c:19
0x00400788 in main (argc=1, argv=0x7efffe94) at buggy_app.c:39
39     char *msg = create_message("Hello Embedded Linux");
Value returned is $10 = 0x404198 "\004\004"
$11 = 0x64 <error: Cannot access memory at address 0x64>
37     /* Test 2 */
38     printf("[TEST2] Creating message\n");
39     char *msg = create_message("Hello Embedded Linux");
40     printf("Message: %s\n", msg);
41
42     /* Test 3 */
43     printf("[TEST3] Integer calculation\n");
40     printf("Message: %s\n", msg);
$12 = 0x404198
#0 main (argc=1, argv=0x7efffe94) at buggy_app.c:40
Breakpoint 5 at 0x400702: file buggy_app.c, line 24.
Continuing.

```

```

Breakpoint 5, calculate (a=100000, b=100000) at buggy_app.c:24
24     int result = a * b;
$13 = 100000
$14 = 100000
22 /* Bug 3: Integer overflow */
23 int calculate(int a, int b) {
24     int result = a * b;
25     printf("calculate(%d, %d) = %d\n", a, b, result);
26     return result;
27 }
25     printf("calculate(%d, %d) = %d\n", a, b, result);
$15 = 1410065408
$16 = 10000000000
$17 = 4
#0 calculate (a=100000, b=100000) at buggy_app.c:25
#1 0x004007b4 in main (argc=1, argv=0x7efffe94) at buggy_app.c:44
r0

```

```

0x186a0      100000
r1           0x186a0      100000
r2           0x186a0      100000
r3           0x540be400    1410065408
r4           0x7efffd08    2130705672
r5           0x40099c      4196764
r6           0x401eac      4202156
r7           0x7efffcd0    2130705616
r8           0x400729      4196137
r9           0x76ffecd8    1996483800
r10          0x0           0
r11          0x401eac      4202156
r12          0x4           4
sp           0x7efffcd0    0x7efffcd0
lr           0x4007b5      4196277
pc           0x40070c      0x40070c <calculate+20>
cpsr        0x200f0030    537854000
fpscr        0x0           0

```

3. Part B — Memory Leak Analysis

Valgrind Memcheck ran on the x86-64 host application, not inside the ARM QEMU guest. The program deliberately allocated record names, score arrays and temporary message buffers without releasing them.

Item	Before fix	After fix
Definitely lost	16,900 bytes / 250 blocks	0 bytes / 0 blocks
Indirectly lost	0 bytes	0 bytes
Heap allocations	352	352

Item	Before fix	After fix
Heap frees	102	352
Error summary	3 errors / 3 contexts	0 errors / 0 contexts

Leak evidence: **create_record** allocates record names (100 allocations; 2,100 bytes) and scores (100 allocations; 2,000 bytes) but **delete_record** only frees the record structure. **process_messages** allocates 50 message buffers (12,800 bytes total) without free. All three classes are reported as definitely lost when the parent record objects have already been freed and no reachable pointer to the children remains.

The corrected source releases `r->name` and `r->scores` before freeing the record, and frees each message buffer after processing. The fixed Valgrind output confirms all heap blocks were freed.

```
==72015== LEAK SUMMARY:
==72015==    definitely lost: 16,900 bytes in 250 blocks
==72015==    indirectly lost: 0 bytes in 0 blocks
==72015==    possibly lost: 0 bytes in 0 blocks
==72015==    still reachable: 0 bytes in 0 blocks
==72015==    suppressed: 0 bytes in 0 blocks
==72015==
==72015== ERROR SUMMARY: 3 errors from 3 contexts (suppressed: 0 from 0)
```

Figure B1. Valgrind leak summary before the fix.

```
==72105== Memcheck, a memory error detector
==72105== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==72105== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==72105== Command: ./leaky_app_fixed
==72105== Parent PID: 71073
==72105==
==72105==
==72105== HEAP SUMMARY:
==72105==    in use at exit: 0 bytes in 0 blocks
==72105==    total heap usage: 352 allocs, 352 frees, 21,124 bytes allocated
==72105==
==72105== All heap blocks were freed -- no leaks are possible
==72105==
==72105== For lists of detected and suppressed errors, rerun with: -s
==72105== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Figure B2. Valgrind summary after the fix.

ARM VmRSS Before/After Comparison

The ARM application **leaky_app_rss_arm** measured memory usage from `/proc/self/status` before record allocations and after processing the allocations. Both measurements were taken in the same process.

Memory metric	Before allocation	After processing	Difference
VmSize	1548 kB	1548 kB	0 kB
VmRSS	156 kB	156 kB	0 kB
VmData	192 kB	192 kB	0 kB

The measured VmRSS did not increase in this short experiment. This does not contradict the Valgrind result of **16,900 definitely lost bytes**: VmRSS measures resident memory pages rather than the number of leaked allocation bytes. Small allocations can use pages already resident in memory.

Evidence: logs/arm_rss_comparison.txt. The earlier single-process observation is also preserved in logs/arm_memory_status.txt.

4. Part C — strace and perf Profiling

On the Ubuntu host, strace -c counted 124 system calls, including 75 write calls (43.24% of measured syscall time). On ARM under QEMU, strace -c counted 120 system calls with 7 error returns; 75 write calls occupied 54.95% of measured syscall time. Library and architecture differences explain distinct syscall inventories. Errors from probes such as access or openat need not indicate application failure.

Host syscall	Calls	Time %	ARM syscall	Calls	Time %
write	75	43.24	write	75	54.95
mmap	14	16.22	mprotect	4	13.59
brk	3	16.22	munmap	2	12.15
openat	4	13.51	mmap2	8	8.62
fstat	5	5.41	rseq	1	1.59

The application wrote 10,000 text lines, yet buffered stdio combined them into just 75 kernel write calls. The percentages above describe time spent in *system calls traced by strace*, not the share of total program runtime.

```
nguyendangquang@nguyendangquang-Bellator:~$ grep -A 15 "LEAK SUMMARY" -/embedded_lab3/logs/valgrind_report.txt
#72815== LEAK SUMMARY:
#72815==    definitely lost: 16,980 bytes in 258 blocks
#72815==    indirectly lost: 0 bytes in 0 blocks
#72815==    possibly lost: 0 bytes in 0 blocks
#72815==    still reachable: 0 bytes in 0 blocks
#72815==    suppressed: 0 bytes in 0 blocks
#72815==
#72815== ERROR SUMMARY: 3 errors from 3 contexts (suppressed: 0 from 0)
nguyendangquang@nguyendangquang-Bellator:~$ grep -A 15 "LEAK SUMMARY" -/embedded_lab3/logs/valgrind_report.txt
#72815== LEAK SUMMARY:
#72815==    definitely lost: 16,980 bytes in 258 blocks
#72815==    indirectly lost: 0 bytes in 0 blocks
#72815==    possibly lost: 0 bytes in 0 blocks
#72815==    still reachable: 0 bytes in 0 blocks
#72815==    suppressed: 0 bytes in 0 blocks
#72815==
#72815== ERROR SUMMARY: 3 errors from 3 contexts (suppressed: 0 from 0)
nguyendangquang@nguyendangquang-Bellator:~$ ls -lh -/embedded_lab3/screenshots/part_b_valgrind/
total 408K
-rw-r--r-- 1 nguyendangquang nguyendangquang 359K Oct 18 08:27 01_memory_leak.png
nguyendangquang@nguyendangquang-Bellator:~$ cat -/embedded_lab3/logs/valgrind_fixed.txt
#72185== Memcheck, a memory error detector
#72185== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
#72185== Using Valgrind-3.22.0 and LLVM/LLVMDB, rerun with -h for copyright info
#72185== Command: ./leaky_app_fixed
#72185== Parent PID: 71873
#72185==
#72185==
#72185== HEAP SUMMARY:
#72185==    in use at exit: 0 bytes in 0 blocks
#72185==    total heap usage: 352 allocs, 352 frees, 21,124 bytes allocated
#72185==
#72185== All heap blocks were freed -- no leaks are possible
#72185==
#72185== For lists of detected and suppressed errors, rerun with: -s
#72185== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
nguyendangquang@nguyendangquang-Bellator:~$ gnome-screenshot -a -f -/embedded_lab3/screenshots/part_b_valgrind/02_memory_leak.png
nguyendangquang@nguyendangquang-Bellator:~$ ls -lh -/embedded_lab3/screenshots/part_b_valgrind/
total 864K
-rw-r--r-- 1 nguyendangquang nguyendangquang 359K Oct 18 08:27 01_memory_leak.png
-rw-r--r-- 1 nguyendangquang nguyendangquang 464K Oct 18 08:28 02_memory_leak.png
nguyendangquang@nguyendangquang-Bellator:~$ cat -/embedded_lab3/logs/strace_summary.txt
# time      seconds  uses/call   calls   errors syscall
-----
43.24  0.000016      0      75         0 write
16.22  0.000006      0     14         0 mmap
16.22  0.000006      2      3         0 brk
13.51  0.000005      1      4         0 openat
5.41  0.000002      0      5         0 fstat
5.41  0.000002      2      1         0 getrandom
0.00  0.000000      0      2         0 read
0.00  0.000000      0      4         0 close
0.00  0.000000      0      4         0 mprotect
0.00  0.000000      0      3         0 munmap
0.00  0.000000      0      2         0 pread64
0.00  0.000000      0      1         0 l access
0.00  0.000000      0      1         0 execve
0.00  0.000000      0      1         0 arch_prctl
0.00  0.000000      0      1         0 set_tid_address
0.00  0.000000      0      1         0 set_robust_list
0.00  0.000000      0      1         0 prlimit64
0.00  0.000000      0      1         0 rseq
-----
00.00  0.000037      0     124         0 total
nguyendangquang@nguyendangquang-Bellator:~$ gnome-screenshot -a -f -/embedded_lab3/screenshots/part_c_performance/01_strace_host.png
```

Figure C1. Host strace -c summary.

```

Environment: QEMU ARM vexpress-a9
Architecture: ARMv7
Tool: strace v4.8
Application: /usr/bin/perf_target_arm

Command:
/usr/bin/strace -c /usr/bin/perf_target_arm

Application Output:
=== Perf Target App - LAB-03 ===
[1] CPU-intensive computation...
Result: 1143803166-974631
[2] Linear search (1000 times)...
[3] I/O intensive (write 10000 lines)...
Done.

System Call Summary:
# time      seconds  usecs/call   calls   errors syscall
-----
54.95 0.002324      33      75      0 write
13.59 0.000624      156      4      0 mprotect
12.15 0.000558      279      2      0 munmap
8.62 0.000396       49      8      0 mmap2
1.59 0.000073        73      1      0 rseq
1.39 0.000064        64      1      0 set_tls
1.31 0.000060        20      3      0 brk
1.24 0.000057        19      3      0 close
1.20 0.000055         9      6      0 lstatx
1.07 0.000049        49      1      0 lseek
0.67 0.000031        31      1      0 set_tid_address
0.65 0.000030        30      1      0 ugetrlimit
0.63 0.000029        29      1      0 clock_gettime64
0.50 0.000023        23      1      0 getrand
0.44 0.000020        20      1      0 set_robust_list
0.00 0.000000         0      2      0 read
0.00 0.000000         0      1      0 execve
0.00 0.000000         0      1      0 access
0.00 0.000000         0      7      0 openat
-----
100.00 0.004593      38     120      7 total

Analysis:
The ARM application executed successfully.

Total system calls: 120
System calls returning errors: 7

The write system call occurred 75 times,
accounting for 54.95 percent of measured syscall time.

Buffered I/O reduced the number of write calls
despite the application writing 10000 lines.

The Ubuntu x86_64 host recorded 124 system calls,
while the QEMU ARM environment recorded 120.

Differences may result from architecture,
system libraries, and runtime environment.
nguyendangquang@nguyendangquang-Bellator:~$ ls -lh ~/embedded_lab3/screenshots/part_c_performance/
total 30K
-rw-rw-r-- 1 nguyendangquang nguyendangquang 306K Oct 10 00:29 01_strace_host.png
-rw-rw-r-- 1 nguyendangquang nguyendangquang 189K Oct 10 00:31 02_strace_arm.png
-rw-rw-r-- 1 nguyendangquang nguyendangquang 549K Oct 10 00:32 03_perf_hotspots.png

```

Figure C2. ARM QEMU strace -c summary.

The perf report shows major application CPU hotspots: **__ieee754_log_fma** (33.12% self overhead in libm.so.6), **heavy_compute** (29.93% self overhead in perf_target) and **log@@GLIBC_2.29** (4.52% in libm.so.6). The expensive sqrt/log work in repeated loops is therefore a strong optimization target. Perf sampling also included some sudo startup overhead, which should not be attributed to the application.

CPU Hotspots - Top 5 Functions

Rank	Function	Shared Object	Overhead
1	__ieee754_log_fma	libm.so.6	33.12%
2	heavy_compute	perf_target	29.93%
3	log@@GLIBC_2.29	libm.so.6	4.52%
4	select_task_rq_fair	[kernel.kallsyms]	0.99%
5	try_charge_memcg	[kernel.kallsyms]	0.99%

The first three hotspots are userspace functions in perf_target or libm. The last two are kernel functions sampled while perf_target was running. Percentages are self-overhead values from the recorded perf profile, which also contains samples from sudo.

```

0.22% perf_target 0x0000000000000000 [.] __crt_lookup_symbol_x
0.22% sudo        libc.so.6      [.] __open64_nocancel
nguyendangquang@nguyendangquang-Bellator:~$ gnome-screenshot -a -f ~/embedded_lab3/screenshots/part_c_performance/03_perf_hotspots.png
nguyendangquang@nguyendangquang-Bellator:~$ ls -lh ~/embedded_lab3/screenshots/part_c_performance/
total 1.1M
-rw-rw-r-- 1 nguyendangquang nguyendangquang 306K Oct 10 00:29 01_strace_host.png
-rw-rw-r-- 1 nguyendangquang nguyendangquang 189K Oct 10 00:31 02_strace_arm.png
-rw-rw-r-- 1 nguyendangquang nguyendangquang 549K Oct 10 00:32 03_perf_hotspots.png
nguyendangquang@nguyendangquang-Bellator:~$ grep -E "Overhead|__ieee754_log_fma|heavy_compute|log@@GLIBC" ~/embedded_lab3/logs/perf_hotspots.txt | head -15
# Overhead Command Shared Object Symbol IPC [IPC Coverage]
33.12% perf_target libm.so.6 [.] __ieee754_log_fma - -
29.93% perf_target perf_target [.] heavy_compute - -
4.52% perf_target libm.so.6 [.] log@@GLIBC_2.29 - -
nguyendangquang@nguyendangquang-Bellator:~$ gnome-screenshot -a -f ~/embedded_lab3/screenshots/part_c_performance/03_perf_hotspots.png

```

Figure C3. Self-overhead hotspots filtered from the perf report.

perf stat showed task-clock 28.42 ms and elapsed wall time 0.030324420 s for the profiled command (including sudo startup). Hybrid-core events were listed separately for cpu_atom and cpu_core. On QEMU ARM, the guest time command reported approximately 0.44 s elapsed. This is not an architecture benchmark: emulation and differing measurement methods limit direct comparisons.

```
nguyendangquang@nguyendangquang-Bellator:~$ ls -lh ~/embedded_lab3/screenshots/part_c_performance/03_perf_hotspots.png
-rw-rw-r-- 1 nguyendangquang nguyendangquang 106K Oct 10 00:32 /home/nguyendangquang/embedded_lab3/screenshots/part_c_performance/03_perf_hotspots.png
nguyendangquang@nguyendangquang-Bellator:~$ cat ~/embedded_lab3/logs/perf_stat.txt
# started on Fri Oct 9 22:11:14 2026

Performance counter stats for 'sudo -u nguyendangquang ./perf_target':

      28.42 msec task-clock
    103,603,833   cpu_atom/cycles/                (94.10%)
     17,535,583   cpu_core/cycles/                (20.31%)
    419,398,839   cpu_atom/instructions/            (94.10%)
     23,888,962   cpu_core/instructions/            (20.31%)
      235,774     cpu_atom/cache-misses/            (94.10%)
       144,595     cpu_core/cache-misses/            (20.31%)
      58,888,531   cpu_atom/branches/                (94.10%)
       4,754,915   cpu_core/branches/                (20.31%)
        111,754   cpu_atom/branch-misses/            (94.10%)
         122,461   cpu_core/branch-misses/            (20.31%)

    0.030324420 seconds time elapsed

    0.004351000 seconds user
    0.005439000 seconds sys

nguyendangquang@nguyendangquang-Bellator:~$ gnome-screenshot -a -f ~/embedded_lab3/screenshots/part_c_performance/04_perf_stat.png
```

Figure C4. Performance counters and hybrid-core events.

Proposed optimizations: reduce expensive logarithm evaluations where mathematically valid; benchmark compiler optimization and loop unrolling; improve contiguous data access/cache locality; or vectorize suitable arithmetic on supported targets. Validate output precision and measure changes with repeated tests rather than assuming any speedup.

5. Part D — Real-Time Latency Analysis

The host general-purpose kernel was tested with cyclicttest: four SCHED_FIFO threads, priority 80, nominal 1000 μ s interval, and 10,000 loops per thread. The stressed run also used stress-ng with four CPU, two I/O and two VM workers.

Thread	No stress Min/Avg/Max (μ s)	Stress Min/Avg/Max (μ s)
T0	0 / 0 / 139	1 / 1 / 592
T1	0 / 1 / 240	1 / 1 / 189
T2	0 / 1 / 244	1 / 1 / 926
T3	0 / 1 / 554	1 / 1 / 511
Highest observed Max	554	926

The largest observed maximum increased from 554 μ s to 926 μ s, a difference of 372 μ s (about 67.1%) in these two experiments. This is a comparison between no-active-stress and stressed measurements on a standard kernel, **not** a measured PREEMPT_RT versus standard-kernel comparison.

```

guyendangquang@guyendangquang-Bellator:~$ cat ~/embedded_lab3/logs/realtime_analysis.txt
LAB03 - Real-Time Latency Analysis
Student ID: SE201072

HOST ENVIRONMENT
Kernel: 7.0.0-34-generic
Architecture: x86_64
Preemption: PREEMPT_DYNAMIC
PREEMPT_RT: Disabled

TEST CONFIGURATION
Tool: cyclicttest
Threads: 4
Priority: 80
Interval: 1000 us
Loops: 10000 per thread

STANDARD KERNEL - WITHOUT ACTIVE STRESS
Thread 0: Min=0 Avg=0 Max=139 us
Thread 1: Min=0 Avg=1 Max=240 us
Thread 2: Min=0 Avg=1 Max=244 us
Thread 3: Min=0 Avg=1 Max=554 us

Overall maximum observed latency: 554 us

STANDARD KERNEL - WITH STRESS-NG
Stress workload:
2 CPU workers
2 IO workers
2 VM workers
128 MB per VM worker
Duration: 60 seconds

Thread 0: Min=1 Avg=1 Max=592 us
Thread 1: Min=1 Avg=1 Max=189 us
Thread 2: Min=1 Avg=1 Max=926 us
Thread 3: Min=1 Avg=1 Max=511 us

Overall maximum observed latency: 926 us

COMPARISON
Without stress: 554 us
With stress: 926 us
Difference: 372 us
Increase: approximately 67.1 percent

ANALYSIS
The measured maximum latency increased under system stress.
Maximum latency is important because a large delay may cause
a real-time task to miss its deadline.

PREEMPT_RT COMPARISON
PREEMPT_RT is designed to improve scheduling predictability
and reduce worst-case latency.

The current Ubuntu kernel does not enable PREEMPT_RT,
therefore, no experimental PREEMPT_RT measurements
were performed in this lab.

CONCLUSION
PREEMPT_RT is generally more suitable for applications
requiring strict timing constraints, such as robot control.
However, meeting a specific deadline must be verified
through testing under realistic worst-case conditions.
guyendangquang@guyendangquang-Bellator:~$ gnome-screenshot -a -f ~/embedded_lab3/screenshots/part_d_realtime/01_cyclictest_results.png

```

Figure D1. Recorded cyclicttest results and stress workload.

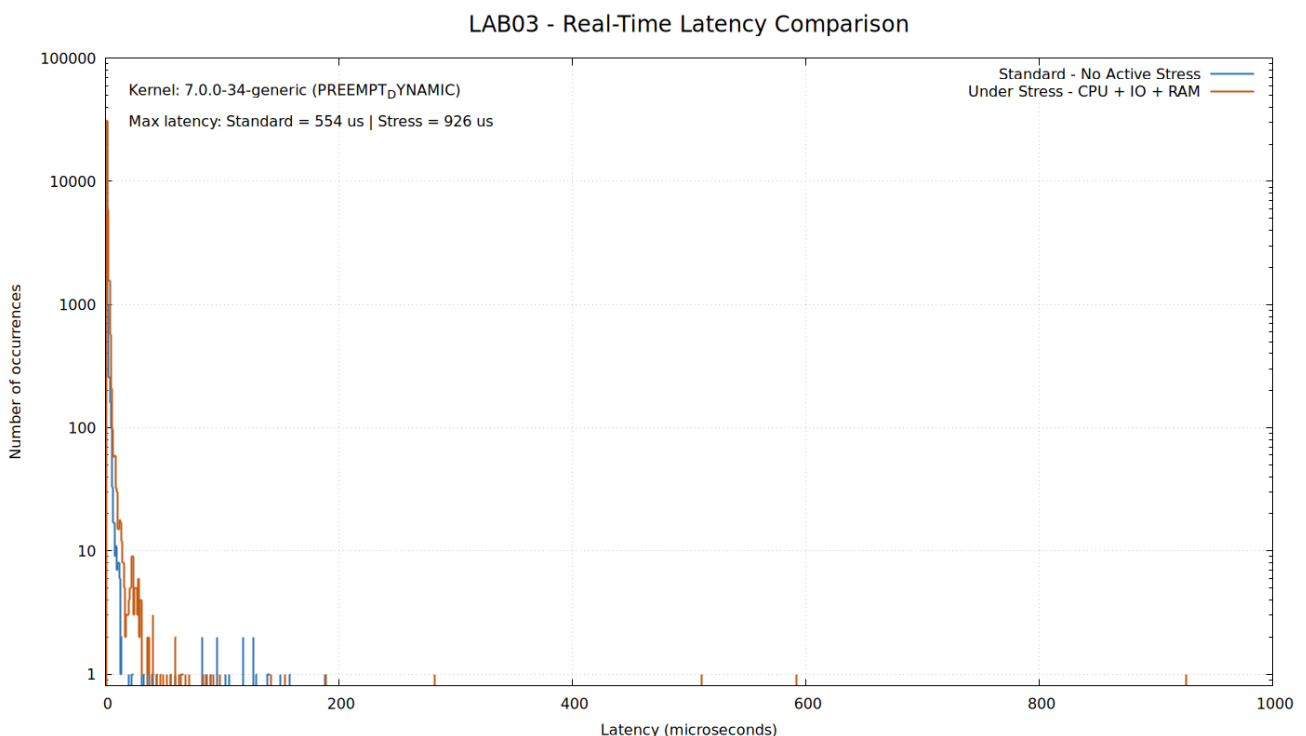


Figure D2. Histogram of measured latency (gnuplot).

Histogram limitation: the baseline histogram collected buckets only through 199 μ s, so higher-latency baseline observations are omitted from the histogram curve; use the cyclicttest maxima above for the full observed maximum. The plot uses a logarithmic frequency scale and compares the two host workloads.

Maximum latency matters more than average for a deadline-sensitive robot because a rare large delay can invalidate a control cycle even when typical delays are small. PREEMPT_RT generally reduces long non-preemptible regions via threaded interrupts and preemptible locking, but a $<500 \mu$ s response requirement must be validated on target

hardware under realistic worst-case loads. In this standard-kernel run, observed maxima already exceeded that threshold.

6. Answers to the Five Analytical Questions

Question 1 — GDB remote next and protocol

The host GDB client encodes a resume/step request via the GDB Remote Serial Protocol (RSP), carried over TCP to the guest gdbserver at port 1234. gdbserver controls the debuggee using target OS tracing facilities and single-steps or runs to the next source-level stop, possibly employing a temporary breakpoint. When the target stops, gdbserver sends a stop-reason packet back; GDB queries registers, memory and symbols and displays the next source line. GDB multiarch understands ARM instruction/register conventions while the host itself is x86-64; a host-only debugger may not support that target. Our observed ARM register dump and remote breakpoints demonstrate this workflow.

Question 2 — Definitely vs indirectly lost

Definitely lost describes heap memory without any pointer that permits reaching the allocation. Indirectly lost describes blocks only reachable through another leaked heap block (for example a leaked parent object pointing to nested allocations). In our leaky_app the containing Record object was freed, leaving no live owning pointer to its name and scores: Valgrind reported these child allocations as definitely lost. Valgrind also diagnoses invalid reads/writes, use of uninitialized values and invalid or double frees. Our pre-fix run found 16,900 definitely lost bytes and 0 indirectly lost.

Question 3 — heavy_compute optimization

First, reduce or reorganize transcendental operations: log and sqrt dominate the sampled hot spots (log-related functions appear at 33.12% and 4.52%). If application mathematics allows lookup tables, approximation, caching of repeated values or algebraic simplification, repeated work can be reduced. Second, improve loop execution by testing compiler optimization flags, safe loop unrolling and contiguous memory access; on ARMv7, NEON can accelerate vectorizable arithmetic, though standard NEON does not directly provide a single vector log instruction, so vector math would require an appropriate approximation/library. Hypothetical improvements such as 10–30% for careful loop changes or up to ~2x for portions made vectorizable are not predictions for this exact code; benchmark and compare numeric error. Gains are limited by the proportion of runtime in the optimized function.

Question 4 — Why Max is much higher than Avg

Interrupt handlers can temporarily delay a real-time thread; PREEMPT_RT moves most IRQ handling into schedulable threads, allowing appropriately prioritized tasks to run first. Kernel critical sections and locking can block preemption; PREEMPT_RT replaces many spinlocks with priority-inheritance capable sleeping locks and shortens unpreemptible execution. Memory management, page faults and background kernel activity can introduce bursts; RT improves preemptibility but cannot eliminate major page faults, so mlockall, prefaulting and resource isolation may still be necessary. Our Avg values were about 0–1 μ s while Max reached 554 and 926 μ s, illustrating why the tail matters.

Question 5 — Tool selection for four failures

(a) Random crash after hours: GDB (ideally collect a core dump and use a reproducible test case) to inspect stack, values and state; the lab found array OOB, UAF and overflow using ARM remote debugging. **(b) Application gradually slows:** Valgrind Memcheck first to identify memory leaks and repeated allocations; our 16,900-byte leak disappeared after fixing 250 blocks; perf can then investigate CPU costs. **(c) Intermittent control-deadline miss:** cyclictst for wake-up latency distributions and spikes under representative load; here maxima exceeded 500 μ s. **(d) One function slows the system:** perf record/report to identify the hot function; here heavy_compute and libm logarithms dominated application samples. strace complements perf when excessive system calls or I/O are

suspected.

7. Difficulties, Resolutions and Conclusion

Challenge 1 — ARM execution environment: an ARM ELF cannot execute directly on the x86-64 Ubuntu host (Exec format error). We cross-compiled with arm-linux-gnueabi-gcc, placed the executable and required ARM shared libraries in the guest rootfs, repacked initramfs and booted QEMU.

Challenge 2 — Remote debugging connectivity: the minimal QEMU guest did not resolve an empty gdbserver listen address reliably. Starting gdbserver on 0.0.0.0:1234 and forwarding TCP port 1234 from the host resolved connection problems.

Challenge 3 — Profiling privileges: the host restricted perf counters; using sudo allowed profiling, although sudo contributes some overhead that must be distinguished from perf_target. QEMU ARM did not initially include strace, so the armhf package was extracted and its binary copied into initramfs.

Challenge 4 — Real-time interpretation: the host used PREEMPT_DYNAMIC rather than PREEMPT_RT. We recorded standard-kernel no-stress/stress results, described PREEMPT_RT theoretically, and avoided inventing RT measurements.

Conclusion: Remote GDB exposed three memory/arithmetic logic errors; Valgrind traced and confirmed repair of all intentional leaks; strace and perf separated syscall behavior from CPU hotspots; and cyclictst quantified potentially deadline-breaking latency spikes. The appended raw evidence and screenshot files make the results traceable.

Appendix — Evidence Files

Raw evidence included in submission: logs/gdb_session.txt, logs/valgrind_report.txt, logs/valgrind_fixed.txt, logs/arm_memory_status.txt, logs/strace_summary.txt, logs/arm_strace_summary.txt, logs/perf_report.txt, logs/perf_hotspots.txt, logs/perf_stat.txt, logs/hist_standard.txt, logs/hist_stress.txt, logs/realtime_analysis.txt, logs/system_environment.txt, logs/latency_histogram.png, and 10 screenshot PNG files grouped by Part A–D.

The recorded data are individual observations under specified conditions and are not worst-case guarantees or hardware-independent performance claims.